

Verifying Thermal Side Channels of Encryption Algorithms on FPGA Final Report

Xudong Dai
Tufts University

Shutong Zhang
Tufts University

Hongyue Jin
Tufts University

Abstract

Thermal side channels have emerged as a promising alternative method to leak sensitive data. However, many studies on breaking encryption algorithms through thermal side channels rely on prior knowledge and there is limited research on how to identify encryption algorithms on an FPGA board using side-channel analysis. Suppose it is feasible to identify the encryption algorithm running on an FPGA board merely with the thermal side channel, a complete set of cracking methods can be applied to crash the encryption. Additionally, countermeasures can be studied to prevent this type of comprehensive attack from being applied in practice.

Therefore, in order to effectively integrate the complete set of methods and develop countermeasures against such attacks, we first studied the thermal characteristics of encryption algorithms running on an FPGA. In this paper, we introduce the possible thermal characteristics of different encryption algorithms and innovatively propose parameters for measuring their thermal characteristics at the algorithm level. Additionally, we extracted thermal feature parameters of different encryption algorithms running on an FPGA under limited resource conditions and achieved an accuracy of 90.4% in identifying them using an artificial neural network, further validating the effectiveness of the algorithm-level thermal feature parameters. This provides a feasible basis for ultimately breaking and protecting various types of FPGA encryption algorithms.

1 Introduction

As computing systems increasingly rely on hardware acceleration to improve performance and energy efficiency, Field Programmable Gate Arrays (FPGAs) have become a popular option for large-scale data centers. For instance, Amazon offers FPGA instances in its EC2 service, which enables customers to accelerate with FPGAs in its cloud computing environment [3]. Additionally, Microsoft utilizes FPGAs extensively in their data centers to perform various tasks ranging from web searches and network cryptography to machine learning [2]. Similarly, Baidu accelerates deep neural networks in their data centers with FPGAs [7]. Moreover, hardware vendors such as Intel and Xilinx have introduced heterogeneous System-on-Chip (SoC) designs that integrate processing cores and FPGA fabric in one silicon DIE. These FPGA-based SoCs are expected to be utilized in mobile and embedded applications.

Many researchers have identified multiple vulnerabilities in various encryption algorithms, along with suggested countermeasures to mitigate these attacks. Specifically, a power analysis attack on an RSA crypto module can easily be executed on an FPGA using the RO-based FPGA power monitor [11]. Power traces from two well-known public datasets, DPA contest V2 [10] and AES_HD [8], captured from the Xilinx Virtex-5 FPGA series, reveal significant vulnerabilities. Additionally, the PRESENT block cipher proposed by Bogdanov et al. at CHES 2007 has been shown to be vulnerable to a statistical saturation attack [5], which extracts information from non-uniform ciphertext distributions, acting as dual saturation attacks. Despite the extensive discussion of vulnerabilities in different encryption algorithms, little effort has been made to identify which algorithm is running on the FPGA.

The thermal side-channel attack has received limited attention due to its complexity and noisy traces [1]. However, in the case of an FPGA, extracting thermal data is easier than that in a traditional data center server, as the temperature of each module can be easily observed using infrared imaging

equipment without the cooling system during the observation.

To investigate the potential direct information leakage in encryption algorithms running on FPGA through thermal radiation, this paper aims to propose a study that involves the use of a thermal camera to capture infrared photos of the running FPGA board, with the aim of analyzing and identifying possible workloads. Power curves are also collected to expand the scope of the project. However, due to experimental limitations, only the relevant thermal feature data at the RTL (Register Transfer Level) can be extracted and the recording and analysis of the thermal characteristics of each module of the FPGA has not been completed. Despite this, it is still possible to identify the encryption algorithm with an accuracy rate of 90.4%, as demonstrated by our implemented neural network. Furthermore, we discuss the probability of conducting the attack automatically based on the workloads running on the board and investigate the possibility of mitigating the thermal characteristics of different encryption algorithms to defend against such attacks.

2 Thermal signature analysis for different algorithms

In this chapter, we will examine and evaluate the feasibility of detecting thermal side channels in encryption algorithms on FPGA at the algorithm level. Our aim is to analyze some representative encryption algorithms across different levels, including the asymmetric encryption algorithm, RSA, the symmetric encryption algorithm, AES, and CKKS, the homomorphic encryption algorithm as one of those post-quantum encryption algorithms. To better compare the recognition effects of encryption algorithms within the same type, we have also included PRESENT and Serpent, which are both symmetric encryption algorithms. By selecting these representative algorithms, we can thoroughly investigate the thermal characteristics of different types of encryption algorithms and further verify the effectiveness of our proposed method.

2.1 Asymmetric Encryption (RSA as the example)

RSA, devised by Ron Rivest, Adi Shamir, and Leonard Adleman, in 1978, is one of the most famous public-key cryptography algorithms. RSA works as follows: to encrypt a message m in the following modular exponentiation operation $c = m \bmod(n)$ is performed, where e , n , and c represent the public key, the modulus, and the ciphertext, respectively. The message can be retrieved from the ciphertext using a similar operation, i.e., $m = c^d \bmod(n)$, where d is the private key. The modular exponentiation is computed using square and multiply operations (see fig.1).

```

1: INPUT  $(M, k, n)$ ; where  $M$  presents the text,  $k$  the key
   which can be represented by its binary representation as
    $k_0 \cdots k_{l-1}$  where  $k_j \in \{0, 1\}$ , and  $n$  the modulus.
2: OUTPUT  $(R_0)$ ; where  $R_0 = M^k \bmod(n)$ 
3:  $R_0 = 1$ 
4:  $R_1 = M$ 
5: for  $j = 0 \leq l - 1$  do
6:    $R_0 = R_0^2 \bmod(n)$ 
7:   if  $k_j = 1$  then
8:      $R_0 = R_0 R_1 \bmod(n)$ 
9:   end if
10: end for
11: return  $R_0$ 

```

Figure 1: Pseudo code of RSA algorithm

The thermal profile of RSA highly depends on the generated key. Generating speaking, the temperature is proportional to the key size. The bigger the key is, the hotter the processor will be. Besides, RSA is a compute-intensive algorithm, then we suppose the temperature curve of the processor could be high and flat, while the memory may be apparently cooler than the processor.

2.2 Symmetric Encryption

Symmetric encryption algorithms refer to those using the same key for both encryption and decryption, and they are generally faster and require less processing power than asymmetric encryption algorithms. They can be divided into two categories: block ciphers, which encrypt data in fixed-size blocks, and stream ciphers, which encrypt data in a continuous stream.

Among all the symmetric encryption algorithms, their differences in power and thermal features are quite apparent. Here we analyze their characteristics respectively.

2.2.1 AES

AES, or Advanced Encryption Standard, was published in 2001 as the next generation encryption standard. It is one of the safest and most popular symmetric encryption algorithms. Its pseudo code is shown in Figure 2.

AES operates on a 4x4 matrix each time. For each matrix, it has 4 kinds of operations combined in the order shown in fig.2. For AddRoundKey, each byte of the state is combined with a byte of the round key using bitwise xor. For SubBytes, it is a non-linear substitution step where each byte is replaced with another according to a lookup table. For ShiftRows, it is a transposition step where the last three rows of the state are shifted cyclically a certain number of steps. For MixColumns, it is a linear mixing operation which operates on the columns of the state, combining the four bytes in each column. In

general, a full encryption will be done 10, 12 or 14 rounds of these 4 operations in order.

```

Cipher(byte in[4*Nb], byte out[4*Nb], word w[Nb*(Nr+1)])
begin
  byte state[4,Nb]

  state = in

  AddRoundKey(state, w[0, Nb-1])          // See Sec. 5.1.4

  for round = 1 step 1 to Nr-1
    SubBytes(state)                        // See Sec. 5.1.1
    ShiftRows(state)                       // See Sec. 5.1.2
    MixColumns(state)                      // See Sec. 5.1.3
    AddRoundKey(state, w[round*Nb, (round+1)*Nb-1])
  end for

  SubBytes(state)
  ShiftRows(state)
  AddRoundKey(state, w[Nr*Nb, (Nr+1)*Nb-1])

  out = state
end

```

Figure 2: Pseudo code of AES algorithm

It is obvious that except AddRoundKey incurs bitwise operations on the processor, all the others are reordering the bytes. Therefore, the pressure is mainly on I/O and memory. Based on this observation, when compared to RSA, the power consumption curve of AES could be more fluctuating than that of the RSA algorithm, with higher values at the peak and lower values at the valley. Since its I/O is much busier than that of RSA, the temperature gap between the processor and memory could be narrower compared to the RSA.

2.2.2 PRESENT

PRESENT is a lightweight block cipher that has faster encryption and decryption speed than RSA and AES. It operates on a 64-bit matrix with 80 or 128 bit keys. As the pseudo code shown in fig.3, it mainly consists of 32 layers of 3 functions, addRoundKey, SBoxLayer and pLayer. addRoundKey is similar, doing xOR operations with input and key. SBoxLayer will do 4bit to 4 bit substitution, and pLayer will do bit permutation.

```

generateRoundKeys()
for i = 1 to 31 do
  addRoundKey(STATE, Ki)
  sBoxLayer(STATE)
  pLayer(STATE)
end for
addRoundKey(STATE, K32)

```

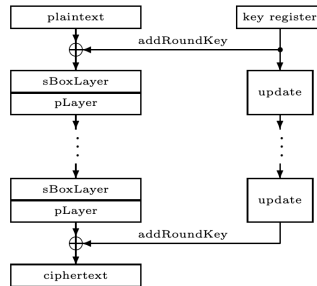


Figure 3: Top-level pseudo code of PRESENT algorithm

Its power consumption curve could be similar to that of the AES algorithm, but the power at the energy peak is slightly lower since all the operations are simpler. The temperature gap between processor and memory could be smaller than that of AES.

2.2.3 Serpent

Serpent is a powerful block cipher that has higher security, but the price is that it is slower than other algorithms. Similar to the previous ones, Serpent works on the input with 32 rounds of 4 operations, addRoundKey, SBoxLayer, pLayer and dLayer, which do key mixing, substitution, permutation and diffusion respectively just like AES.

Since Serpent has more rounds, it may cost more energy compared to AES. Its power consumption curve might be similar to that of the AES algorithm but the fluctuation will be more obvious than AES. Therefore, the temperature gap between processor and memory could be bigger than that of AES, but still smaller than that of RSA.

2.3 Homomorphic Encryption

Homomorphic encryption is a form of encryption that allows computations to be performed on the ciphertext. In other words, it enables computations to be carried out on encrypted data without the need to decrypt it first. This feature makes it highly applicable when computing sensitive data since it balances confidentiality and privacy.

2.3.1 CKKS

CKKS (Cheon-Kim-Kim-Song) is a homomorphic encryption scheme, which has gained popularity for its ability to perform homomorphic operations on complex data types, such as polynomials and matrices, with low overhead. It is particularly well suited for machine-learning applications.

In terms of the power curve, CKKS has a flat and constant one compared to those of RSA and AES without significant spikes. Regarding the temperature distribution, it will generate lower temperature differentials between processor and memory since it mainly performs fixed-point arithmetic operations with a small modulus, which reduces power consumption and heat dissipation. Its power and thermal features will be entirely different.

2.4 Flops/Iops Ratio

As we discussed the different working features of these algorithms, the Flops/Iops ratio of these algorithms are also different according to the features. The difference can reflect the temperature gap between the processor and memory.

Generally speaking, this ratio of symmetric algorithms is smaller, between 10^2 - 10^4 . However, this ratio of homomorphic algorithms will be much higher, which is about 10^6 - 10^9 .

3 Methodology

In this chapter, we will provide a detailed analysis and discussion of our experimental process. Firstly, we will showcase our initial research plan, including the selection of equipment and platforms, data collection, and verification of conclusions. However, due to a series of difficulties encountered at the initial stage of the experiment, including the lack of equipment and software algorithms, as well as lack of experience, we adjusted our plan according to the real situation. On this basis, a new research plan will be expounded, and a comparison between the old and new experimental plans will be made, as well as the rationale behind using RTL to replace the final infrared camera imaging. We also present our experimental results and perform the analysis to verify the feasibility of using the thermal side channels to identify the encryption algorithms running on the FPGA board.

3.1 Original Plan

The following steps were taken to verify thermal side-channel attacks on FPGA boards:

Step 1 - Collection of power consumption and thermal data: The power consumption and thermal data will be collected by running the encryption algorithms on the FPGA board. An oscilloscope will be used to measure power consumption [9], while a thermal camera will be used to capture the thermal/infrared images [6].

Step 2 - Analysis of power and thermal data: The power and thermal data will be analyzed to extract distinguishable patterns of different encryption algorithms. Software tools will be used to process the data and look for certain features on board, including power peaks, thermal distribution, and temperature change and fluctuation.

Step 3 - Identification of encryption algorithm: After analyzing the power and thermal data, attempts will be made to identify the encryption algorithm by comparing the data with known patterns for different encryption algorithms. If there are distinguishable patterns, the encryption algorithm could be identified with high accuracy.

Step 4 - Validation of results: The results were validated by repeating the experiment with different parameters, such as key length and input data size, to ensure that the thermal side-channel attack was consistent and reliable [4].

3.2 Initial experimental design

The original plan for the project involved several steps. Firstly, a selection of encryption algorithms, including both traditional and post-quantum, were compiled using Vivado and Vitis and run on the FPGA board. During this process, a power meter and thermal camera were utilized to measure power and temperature curves. These curves were then analyzed to identify distinctive features of the algorithms, with the goal

of detecting which algorithm was running during subsequent runs. If time is permitted, the plan also included incorporating neural networks to automate the detection process.

During the project, several challenges were encountered which impeded progress. The first issue was the inability to test workloads on a real FPGA board due to the lack of a driver for the Zedboard produced in 2014. This board could not be recognized by Vivado on Windows laptops or the Linux desktop at the laboratory. Additionally, two other boards that were acquired did not have the necessary power cables, which prevented program execution on a real board.

The second issue was the inability to run emulation through Vitis. While some implementations of encryption algorithms could be compiled through Vitis, the emulation could not be run due to the lack of necessary root privilege and disk space, or the unavailability of the required function on Vitis 2022.1 for Windows.

The third issue was the inability to complete the compilation of a post-quantum algorithm through Vitis in a timely manner. Repositories on GitHub were browsed, but most of them required third-party number-theory libraries, which must be compiled before the interface can be used. The migration of these libraries to an FPGA platform would require additional time and resources, which were unavailable. Thus, the focus was shifted to traditional algorithms that were easier to migrate.

The fourth issue we faced during the training of artificial neural networks was the lack of sufficient data sets, due to the issues above. According to our initial research plan, we anticipated building two sets of neural networks, one for recognizing power curves of different encryption algorithms and workloads running on the FPGA board, which includes the total power consumed during program execution and power fluctuations of each component. The data in this part is characterized by a relatively small amount, and each group of data should have different heat generation patterns based on different workloads. Therefore, we plan to use relatively basic fully connected neural networks to recognize this part of the data.

3.3 Solution

In this section, we go through the whole process of our project, discussing what we have done and what we have achieved.

We finally turn to the Vivado HLS tool installed on the Tufts Linux server to synthesize and emulate some encryption algorithms, including AES, Serpent, PRESENT, and RSA. Through the syntheses, we can observe the area and latency of these algorithms. Besides, if we modify the optimization in the C++ code, it can be helpful to observe the operations on which this algorithm highly relies.

In order to expand our datasets, we introduced some example designs. By comparing the encryption and non-encryption workloads, we wish we can find more distinct thermal features

of these encryption algorithms.

We also explore the possibility of extending neural networks to automate the whole process. Neural networks can help improve the accuracy and efficiency of recognizing the algorithms, which could be of great benefit.

3.4 Why Vivado HLS

In order to collect the data, we choose to get the aforementioned workloads from open-source platforms. To run these workloads, we choose to use Vivado HLS to synthesize executables compatible with the FPGA board.

Vivado HLS (High-Level Synthesis) is a tool designed by Xilinx to translate the C/C++ code into RTL code that can be implemented on the FPGA. It is designed to enable hardware designers to accelerate the development of FPGA systems with their software expertise, which can significantly reduce the time and cost when developing new products in the ever-changing market. However, its high-level features bring certain disadvantages simultaneously. The designers may lose control over low-level details, and while difficult for them to reach higher clock frequencies. It is still important to hire experienced designers to optimize the low-level details for better performance and energy efficiency.

In this case, Vivado HLS can help us simplify the developing process. Though all team members had experience in Verilog, the time is still limited when translating the algorithm into Verilog. Without Vivado HLS, we cannot collect enough data from so many workloads in such a short time. Besides, considering the purpose is to extract the general thermal features of these algorithms, it is unnecessary to use the maximum optimized versions of the workloads. Considering the 2 reasons, Vivado HLS is the best tool we can use.

3.5 Why Analyze Optimizations

When walking through the code we ran, we found there are plenty of precompile optimization instructions, such as `#pragma HLS pipeline` and `#pragma HLS unroll`. These 2 instructions are used to accelerate the loop. The pipeline is used to break up the loop into stages such that each stage can start processing the next iteration of the loop while the previous stage is still processing the current iteration.

The unroll is used to enable loop unrolling, which refers to replicating the loop body multiple times to reduce loop overhead and increase parallelism.

We choose to analyze the optimization for 2 reasons. First, we need to expand our dataset. We merely have 8 workloads. Since it is unable to run either the real board or the emulation, we merely have one implementation for each workload and one data point. If we can set several optimization levels for each of the workloads, we are able to include more data points into the dataset for our neural network.

Besides, in the real world, the thermal features of the same algorithm are related to multiple factors, such as the device, the implementation, and environment temperature, etc. It is practical to include more variants of the same algorithm to improve the accuracy and robustness of our neural network model.

3.6 Why Neural Networks

For the thermal distribution data on the board, we plan to sample and process the frames of the video captured by the infrared camera, and matrix-sampling the entire area of the FPGA board to form the temperature matrix of the FPGA board frame-by-frame. Due to the introduction of temperature matrix and frame-by-frame sampling over time, the amount of data for this part will be greatly increased. As convolutional neural networks are more suitable for processing matrix data, we hope to introduce them to recognize this part of the data. Some famous CV models, like YOLO will also be tested and compared with our CNN model. If we can obtain two types of data for the thermal features of the FPGA, we can further introduce new neural networks to make final decisions based on the output of the two neural networks mentioned above.

Experiments

In this section, we walk through the whole process of our experiment, list our results and give out our analysis.

3.7 Code Source

We get part of our code from HLS-crypto, which is a repository containing AES, Serpent, and PRESENT algorithms good for vivado_hls synthesis. The RSA algorithm is from HLS-RSA.

In order to unify the comparison for algorithms in HLS-crypto, we took the masked-reconf versions of these algorithms.

3.8 Experiment Setups

We wrote the `run_tls.tcl` file for all these algorithms. In the tcl file, there are 2 key instructions we use, `csynth_design` and `cosim_design` like the following,

```
csynth$ \_ $design
cosim$ \_ $design -rtl vhdl -tool xsim -tra
```

csynth_design stands for “C synthesis design”. This step will synthesize the RTL code from the original C code. **cosim_design** means “Co-simulation design”. At this step, the generated RTL code will be simulated with the testbench to observe the final result.

We picked up xsim as the tool in `cosim_design`. After iterating all the options, xsim is the only one installed on the

Algorithms	Min Latency (cycle)	Max Latency (cycle)
AES	586	586
PRESENT	136	136
Serpent	1173	1173
RSA	1209	17961
fft_single	3195	3195
fft_2ch_int	5001	5001
svd_pairs	15540	15690
2D_conv	6	2070610

Table 1: Latency of these algorithms

server. However, this tool can simply give the latency, which does not provide us with more information.

We also created the proj directory to store the synthesis and simulation results for each algorithm.

We picked up another 4 workloads to compare with the 4 encryption algorithms.

- fft_single, single precision floating point Fast Fourier Transform
- fir_2ch_int: 2-channel FIR filter using integer coefficients.
- svd_pairs: Singular Value Decomposition of pairs of matrices
- 2D_convolution_with_linebuffer: 2D convolution implementation using a line buffer.

We plan to reach 2 goals during the experiments. Compare the area and latency of different algorithms and search for featured patterns. Compare the effectiveness of optimization in the code, like #pragma, to evaluate how optimization may affect the thermal features of algorithms.

The environment steps include Run all 4 algorithms, and compare their area and latencies with Vivado HLS 2019.2. We also run other workloads to compare their working features. Remove all the #pragma HLS pipeline to compare the results. Bring back all #pragma HLS pipeline, then continue to remove #pragma HLS unroll to watch what happens next. Remove #pragma HLS pipeline then watch what happens next and finish the data analysis of the experiments

3.9 Experiments Results & Analysis

The following tables contain some of the vital data generating by some encryption algorithms.

Takeaway #1 We find that the latency is strongly correlated with the area, almost a linear correlation, according to Table.3.9. The more complex the algorithm is, the more area it needs to use, and more latency to finish one single step of computation.

Algorithms	Memory FF	Memory LUT	Instance FF	Instance LUT	FIFO FF	FIFO LUT
AES	114	1058	1106	4170	-	-
PRESENT	68	33	561	963	-	-
Serpent	128	256	11395	10316	-	-
RSA	-	-	33844	26385	-	-
fft_single	-	-	9912	8038	20	136
fft_2ch_int	-	-	349	429	10	80
svd_pairs	0	0	11478	12993	-	-
2D_conv	-	-	1730	2761	55	484

Table 2: Area of these algorithms

Takeaway #2 There is an apparent difference between the symmetric and asymmetric algorithms. For the symmetric algorithms, their min and max latencies are all the same. However, for the asymmetric RSA, the min and max latencies are 15 times different. We think it is the intrinsic property of RSA since the key pairs will lead to different amounts of power and mod operations, which are time-and-power-consuming if we have the chance to verify the presumption.

Takeaway #3 We also noticed the difference in areas of FF, LUT, and FIFO used by these algorithms based on Table.3.9. FF stands for Flip-Flops, which are used to store the intermediate result, while the LUT stands for Look-up Tables, which convert the input to the output to implement the digital logic. The amounts of FF, LUT, and FIFO used by distinct algorithms are also different. We notice that only Serpent uses a similar amount of FF and LUT, while AES has 4 times more area of LUT compared to the FF area. Meanwhile, Serpent uses 2 times more area of LUT than the FF area.

Our RSA does not take any memory area. We suppose that each RSA encryption operation does not take any memory to store the block information. It just needs to compute the mod and power operation over and over again. This algorithm simply needs registers.

We know the working principles of these algorithms are quite different, therefore the differences reflect on the areas. If we can remove the hull of the FPGA chip and take some infrared photos, we may tell the feature of the algorithms by the heat generated by different areas, either LUT, FF, or FIFO.

We here only list the latency of multi-level optimized AES algorithms since its implementation contains more optimizations than the others. There are mainly 2 optimization pre-compile instructions, which are

```
#pragma HLS pipeline
#pragma HLS unroll
```

These 2 operations are used to optimize the loop operation. The ‘pipeline’ will break the loop into multiple smaller independent stages to improve the frequency, while the ‘unroll’ will unfold the loop into several parallel iterations. The first one is good for a step with multiple operations, while the

Optimizations on AES	Min Latency (cycle)	Max Latency (cycle)
Pipeline + Unroll factor = 2	938	938
Pipeline + Unroll factor = 4	736	736
Pipeline + Unroll factor = 8	672	672
Pipeline + Unroll factor = 16	586	586
No Unroll	1275	1275
No Pipeline	563	563
No pipeline nor unroll	75316	75316

Table 3: Latency with different optimization of AES

second is good for one step with one repeated operation.

Takeaway #4 We observe that, simply without pipeline optimization, the latency is even smaller based on Table.3.9. We believe it is because all the operations are serial, then the optimization could have negative effects. Without unroll operation, it is obviously slower since AES has a number of loops with unroll optimization.

Takeaway #5 The ‘unroll’ and ‘pipeline’ operations have amplification effects. The most shocking one is the bare version with no single optimization. The latency increases almost 15 times! Based on this observation, we can tell how important the #pragma optimization is.

Takeaway #6 The different optimization levels of the same algorithm has little affect on the recognition accuracy, which indicates that the intrinsic properties of these workloads decide.

Coming to the neural networks, due to the aforementioned challenges, the encryption algorithms actually running on the FPGA were not obtained, and only a series of RTL reports and corresponding data for specific areas were obtained. As a result, the aforementioned approaches had to be appropriately adjusted. The part of recognizing thermal distribution using a convolutional neural network had to be abandoned. Furthermore, since the RTL reports are more similar to the power curve of the FPGA, we still used fully connected neural networks to process the data. Trained with the existing dataset, the model achieved over 80% accuracy in the final recognition on the test set, as shown in Fig4.

In order to better simulate real-world scenarios, we added Gaussian noise with a standard deviation of 5% to each data group. As a result, the accuracy of the recognition curve, represented by the yellow line in Fig.5, was obtained. Through comparison, we found that the stable recognition accuracy decreased significantly after introducing noise. We attribute this to the lack of sufficient data sets, which leads to lower resistance to noise.

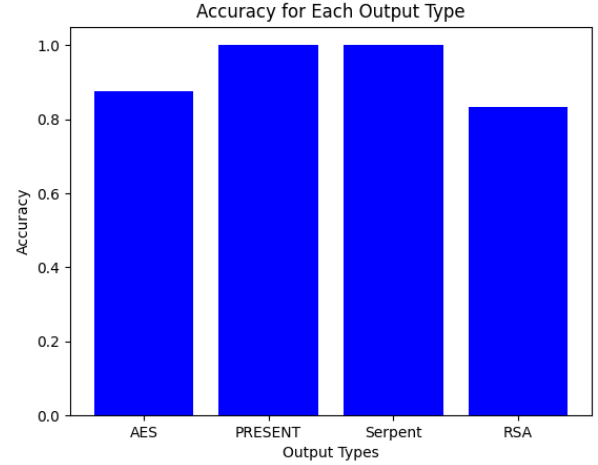


Figure 4: Accuracy of neural network recognition for each algorithm

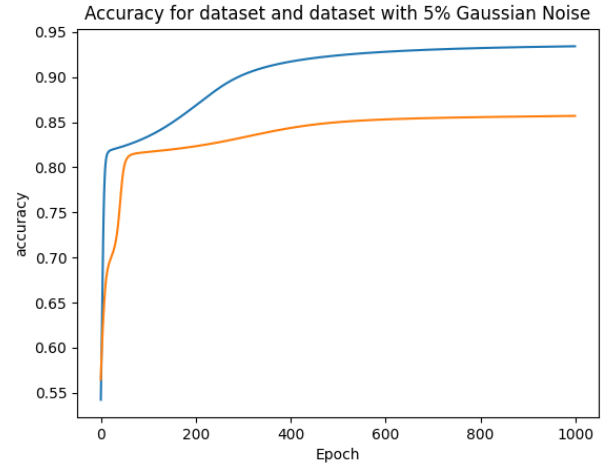


Figure 5: Accuracy of neural network recognition (Blue stands for accuracy for the original dataset, orange stands for the dataset with 5% Gaussian Noise)

4 Conclusion

After walking through all the difficulties, we now collect some interesting observations from the latency and area of these algorithms. The latency and area are linearly proportional. Besides, the area of LUT and FF used by different algorithms may reflect some interesting intrinsic features of the algorithm, which may tell hackers which algorithm it is.

The precompile #pragma instructions play an important role in improving the running speed of the algorithms. Vitis

perhaps doesn't have a strong compiler to automatically optimize the code, therefore the programmers should also take the optimization into consideration at an early stage.

Based on the output of our artificial neural network, we have achieved a decent recognition accuracy despite the limited data. However, the resistance of the recognition model to noise interference is insufficient. If real power curves are introduced, the existing data is obviously insufficient to recognize the use of different encryption algorithms with different workloads under complex conditions.

5 Future Work

At present, our project is in its preliminary stage. We hope to gain valuable insights by simulating the power consumption using Vitis. If we can successfully generate the bitstream and program an FPGA board, we will have the opportunity to validate our theory in the physical world. As time permits, we intend to execute our program on the hardware. In order to do so, we plan to either run Vitis or acquire a new board with an appropriate power cable and driver to connect to our computers. This will be our top priority for future work. It is also our desire to implement certain post-quantum algorithms on an FPGA board. However, as previously stated, the open-source implementations necessitate the use of third-party libraries. We can attempt to develop the requisite stub functions, enabling us to execute these post-quantum algorithms on the FPGA boards.

If we are able to run the algorithms, we will be able to collect sufficient data to train our neural networks. As we have previously discussed, we aim to extract feature data from two aspects: one is to extract and recognize power curves of different encryption algorithms applied to different workloads, and the other is to recognize the temperature images of the FPGA board during operation using an infrared camera. In the recognition process, a CNN neural network will be established and compared with existing well-known CV recognition models. Additionally, since temperature data is heavily affected by the environment and noise, more images captured under different external temperature conditions will also be recorded. Ultimately, a thermal-based encryption algorithm recognition tool for FPGAs, which includes power curves and temperature image recognition, will be designed. In addition to this, a more distant and ambitious task includes research on cracking the existing range of encryption algorithms, integrating their methods, and combining them with our existing encryption algorithm recognition tool, to design a comprehensive tool suite that covers the entire process from data extraction to encryption algorithm cracking.

Another area of focus involves exploring effective counter offensives to obfuscate the thermal features of existing algorithms. For example, we might introduce meaningless instructions into the algorithm. While this could lower the encryption efficiency, it could significantly enhance the algorithm's secu-

rity. To validate the effectiveness of these modified algorithms, we can use the recognition tool we previously developed to identify the thermal characteristics of the modified encryption algorithms.

6 Acknowledgments

We sincerely appreciate the help from Dr. Parnian Mokri. She has always been working to help us deal with all the technical difficulties of Vivado and Vitis, and taught us how to improve our report and project.

We are also grateful to Prof. Mark Hempstead for his guidance and comments on our project.

References

- [1] J. Bouchier, T. Kean, C. Marsh, and D. Naccache. Temperature attacks. *IEEE Security & Privacy*, 7(2):79–82, 2009.
- [2] A. M. Caulfield, E. S. Chung, A. Putnam, H. Angepat, J. Fowers, M. Haselman, S. Heil, M. Humphrey, P. Kaur, J.-Y. Kim, et al. A cloud-scale acceleration architecture. In *2016 49th Annual IEEE/ACM international symposium on microarchitecture (MICRO)*, pages 1–13. IEEE, 2016.
- [3] J. Choi, R. Lian, Z. Li, A. Canis, and J. Anderson. Accelerating memcached on aws cloud fpgas. In *Proceedings of the 9th International Symposium on Highly-Efficient Accelerators and Reconfigurable Technologies*, pages 1–8, 2018.
- [4] K.-S. Chong, J.-S. Ng, J. Chen, N. K. Z. Lwin, N. A. Kyaw, W.-G. Ho, J. Chang, and B.-H. Gwee. Dual-hiding side-channel-attack resistant fpga-based asynchronous-logic aes: Design, countermeasures and evaluation. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, 11(2):343–356, 2021.
- [5] B. Collard and F. X. Standaert. A statistical saturation attack against the block cipher present. In *Topics in Cryptology—CT-RSA 2009: The Cryptographers' Track at the RSA Conference 2009, San Francisco, CA, USA, April 20–24, 2009. Proceedings*, pages 195–210. Springer, 2009.
- [6] M. Hutter and J.-M. Schmidt. The temperature side channel and heating fault attacks. In *Smart Card Research and Advanced Applications: 12th International Conference, CARDIS 2013, Berlin, Germany, November 27–29, 2013. Revised Selected Papers 12*, pages 219–235. Springer, 2014.

- [7] J. Ouyang, S. Lin, W. Qi, Y. Wang, B. Yu, and S. Jiang. Sda: Software-defined accelerator for large-scale dnn systems. In *2014 IEEE Hot Chips 26 Symposium (HCS)*, pages 1–23. IEEE, 2014.
- [8] S. Picek, A. Heuser, A. Jovic, S. Bhasin, and F. Regazzoni. The curse of class imbalance and conflicting metrics with machine learning for side-channel evaluations. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2019(1):1–29, 2019.
- [9] S. Tao, Y. Yu, and E. Dubrova. Fpga based true random number generators using non-linear feedback ring oscillators. In *2018 16th IEEE International New Circuits and Systems Conference (NEWCAS)*, pages 213–216. IEEE, 2018.
- [10] H. Wang and E. Dubrova. Tandem deep learning side-channel attack on fpga implementation of aes. *SN Computer Science*, 2:1–12, 2021.
- [11] M. Zhao and G. E. Suh. Fpga-based remote power side-channel attacks. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 229–244. IEEE, 2018.

7 Appendix: Problems encountered during the project and Data set

Things went quite unsmoothly during our whole process. We encountered many problems that almost killed our project. Here is a brief description of all these problems.

First, we are unable to test the workloads on a real FPGA board, shown in the Picture 1. We initially planned to compile our program, then ran it on the FPGA board. Unfortunately, we cannot find a driver for the Zedboard produced in 2014. The board cannot be recognized by the Vivado either on our Windows laptops or the Linux desktop at the laboratory. That is, the compiled bitstream cannot be run on the board. We got other 2 boards later, but none of them had a proper power cable. After all, we cannot run the program on a real board.

Second, we cannot run emulation through Vitis. We found some implementations of these encryption algorithms which can be compiled through Vitis. We wish we could run the emulation on Vitis so that we can still get some crucial statistics like power consumption. However, this plan finally went to a dead end. If we tried to run the emulation on Windows, Vitis 2022.1 on Windows does support the function. If we turned to the Vitis 2022.1 installed on the Tufts server, it required the root privilege with 80GB disk space. In short words, there is no way for us the run the emulation.

Finally, we cannot finish a post-quantum algorithm compilable through Vitis in a short time, after struggling with the previous problems. We browsed many repos at GitHub, but most of them require some 3rd-party number-theory libraries since these algorithms are designed based on these special mathematical properties. It is unreasonable for those developers to do these libraries again. However, these libraries must be compiled to use before calling their interfaces. We cannot have enough time to migrate these codes to an FPGA platform. Therefore, we have to focus on some traditional algorithms which are easier to migrate.

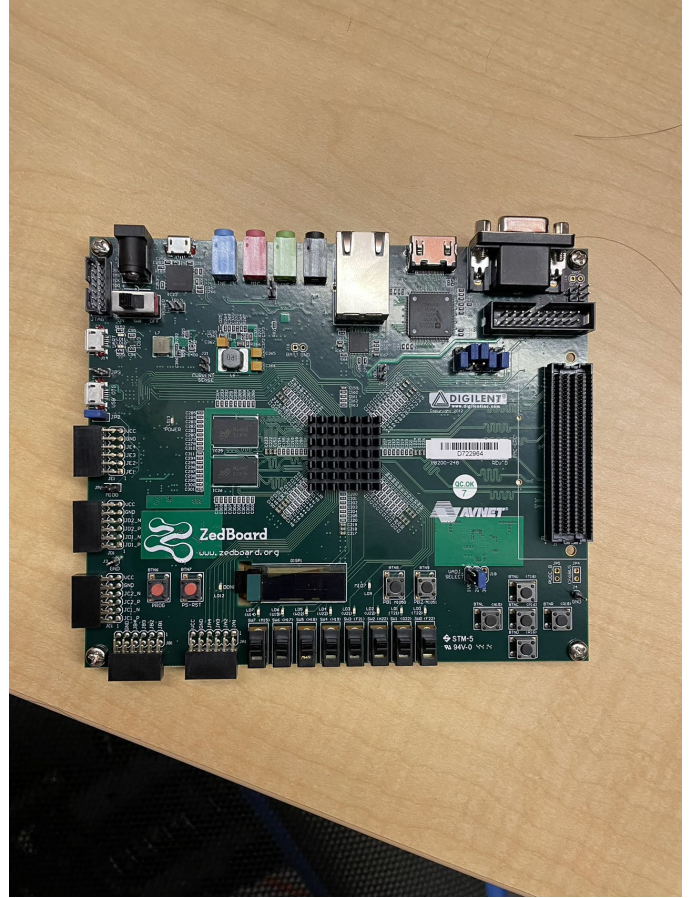


Figure 6: Zedboard we were working on

Table 4: Total Dataset

Algorithms	Min Latency (cycle)	Max Latency (cycle)	Memory FF	Memory LUT	Instance FF	Instance LUT
AES	586	586	144	1058	1106	4170
AES Pipeline + Unroll factor = 2	938	938	144	1058	14767	25229
AES Pipeline + Unroll factor = 4	736	736	144	1058	12444	18415
AES Pipeline + Unroll factor = 8	672	672	144	1058	23382	33178
AES Pipeline + Unroll factor = 16	586	586	144	1058	1106	4170
AES No Unroll	1275	1275	144	1058	10226	16648
AES No Pipeline	563	563	168	1058	1676	5131
AES No Pipeline nor unroll	75316	75316	144	1058	10226	16648
PRESENT	136	136	68	33	561	963
PRESENT Pipeline + Unroll factor = 2	266	266	68	33	609	1095
PRESENT Pipeline + Unroll factor = 4	170	170	68	33	910	1148
PRESENT Pipeline + Unroll factor = 8	154	154	68	33	966	1196
PRESENT Pipeline + Unroll factor = 16	146	146	68	33	1078	1292
PRESENT No Unroll	454	454	68	33	610	785
PRESENT No Pipeline						
PRESENT No Pipeline nor unroll	18517	18517	74	39	3015	2623
Serpent	1173	1173	128	256	11395	10316
Serpent Pipeline + Unroll factor = 2	1178	1178	128	256	9914	7882
Serpent Pipeline + Unroll factor = 4	1176	1176	128	256	10419	8801
Serpent Pipeline + Unroll factor = 8	1173	1173	128	256	11395	10316
Serpent Pipeline + Unroll factor = 16	1173	1173	128	256	11395	10316
Serpent No Unroll	1398	1398	128	256	10248	7311
Serpent No Pipeline						
Serpent No Pipeline nor unroll	95577	95577	128	256	27700	19830
RSA	1209	17961	0	0	33844	26385
RSA Pipeline + Unroll factor = 2	897	17649	0	0	39903	32165
RSA Pipeline + Unroll factor = 4	741	17493	0	0	52021	43725
RSA Pipeline + Unroll factor = 8	663	17415	0	0	76257	66845
RSA Pipeline + Unroll factor = 16	580	17332	0	0	27785	20605
RSA No Unroll	1207	17961	0	0	33844	26385
RSA No Pipeline						
RSA No Pipeline nor unroll						